INTERVENTIONS, SIMPLE BASELINES, AND COST LIMITS IN SPECULATIVE DECODING

Anonymous Authors[1]

ABSTRACT

Speculative-decoding output reproduction, reference compatibility and optimization require different evidence. We examine these distinctions through controlled consumer-system studies. A B570 prefill intervention restores four known deviations without new deviations among 20 controls. An RTX target-continuation intervention is predicted exactly on 43 tasks but repairs one task and introduces another deviation, leaving reference agreement unchanged. A subsequent matched 19-task study finds ordinary draft-free target-only execution reproduces every RTX intervention output, limiting the demonstrated need for the more complex predictor. An offline intervention-selection analysis finds no advantage over the tested simple alternatives. Separate cost evidence shows qualified RTX serialization slower than target-only, conditional 358H overhead-removal ceilings below parity, and M4 Guard slower than fixed speculation. The contribution is a bounded empirical characterization with complete-output contracts, paired effects, strong baselines and explicit negative results, rather than a general repair or new decoding method. These findings identify what the measured stacks support and which optimization claims remain unestablished.

1 INTRODUCTION

Speculative decoding amortizes target computation with cheaper proposals (Leviathan et al., 2023). In a numerical stack, output reproduction, reference compatibility, restoration and cost benefit require different evidence. Prior work establishes batch-shape sensitivity, deterministic verification and heterogeneous performance limits (Yuan et al., 2025; Gond et al., 2026; Chordiya, 2026; Liu et al., 2026b). We examine their interaction through controlled consumer-system studies.

A program can reproduce native outputs without being a necessary or cheaper predictor. An intervention can repair one task and break another while aggregate agreement stays unchanged. A qualified path can lose to ordinary target-only execution. We test these possibilities with explicit output contracts, paired outcomes, simple baselines and separate cost measurements.

On B570/Vulkan, a draft-free program predicts a prefill intervention that restores four known deviations and retains 20 matching controls. On RTX/CUDA, proposal/target programs predict a continuation intervention on 19 exposed HumanEval and 24 exposure-audited MBPP tasks. It restores one task and introduces another deviation: prediction succeeds while the restoration objective fails.

A subsequent matched RTX study changes the interpretation. Ordinary draft-free target-only UB2 reproduces every intervention output on the 19-task cohort, including both changed tasks. The simpler baseline limits the demonstrated need for the complex predictor, without establishing general equivalence. An offline feasibility study finds no supported automatic selector: state differences supply no repair direction, and screening adds work to the sole successful B570 candidate.

Separate cost studies show qualified RTX serialization slower than target-only; conditional 358H overhead-removal ceilings below parity; and a prospective M4 guard slower than fixed speculation. These configuration-specific results retain different timing boundaries and do not rank devices.

The contribution is a bounded empirical record: complete-output intervention tests with restoration/regression accounting; a fresh simple-baseline comparison and negative selection analysis; and qualified cost comparisons with explicit conditional budgets. We introduce no new decoding algorithm or general repair method. The studies show how strong baselines and complete cost accounting constrain the utility assigned to successful prediction and control.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

2 Output Contracts and Decision Criteria

2.1 Reference, reproduction and intervention

Let k denote maximum draft width and UB the target micro-batch capacity. A greedy speculative round verifies proposals, commits the agreeing prefix and a correction or bonus when budget permits, and removes inconsistent suffix state. UB is a capacity; it does not specify actual occupied graph shape or the history of forwards.

Our output contract compares the complete committed generated-ID array and endpoint with a matched target-only greedy reference. Prompt IDs and uncommitted proposals are excluded. A terminal end-of-generation ID (EOG) is inclusive; otherwise the study cap applies. "Complete" describes this bounded output, not all internal states or all untruncated process output. Historical text-path agreement (H) and direct-ID qualification (Q) remain distinct. Neither token equality nor mismatch measures functional code correctness or pass@k.

Write R for the reference, P_0, P_1 for frozen baseline/intervention predictions, and N_0, N_1 for native outcomes. Reproduction $P_j = N_j$ differs from compatibility $N_j = R$. A task is restored if $N_0 \neq R$ and $N_1 = R$, newly divergent if $N_0 = R$ and $N_1 \neq R$, and persistently divergent if both fail. Persistent deviation is an output no-op only when $N_0 = N_1$ as well. Three repetitions test stability, not three independent tasks.

The descriptions are manually constructed programs. B570 transfers target-only prefill, pending-token and sampler state into continuation. RTX separately orchestrates live proposal/target workers and suffix reconciliation using the native path's pinned numerical routines. Native uses its own live draft without supplied answers. Neither description is a learned predictor or independent numerical implementation.

The new simple-baseline study denotes ordinary target-only UB2 by U_2. Reproducing N_1 with U_2 tests whether a simpler existing path suffices for that output objective. It does not require $U_2 = R$, prove identical KV/logits, or establish a cost saving.

2.2 Cost and intervention selection

Let T_b be matched target-only time per output token, C all committed output per round, T_d^{real} the complete draft subsystem, $T_{\text{target,round}}^{\text{real}}$ all target work, and T_x residual charged work. Then

$$S = \frac{T_b C}{T_d^{\text{real}} + T_{\text{target,round}}^{\text{real}} + T_x} \tag{1}$$

$$= \frac{C}{D_{\text{real}} + R_{\text{target}} + X}. \tag{2}$$

These accounting identities are established tools (Kim et al., 2026), not new profitability theory. Target-forward proxies omit drafting and integration, and incompatible measurements cannot isolate a component by subtraction. Compatibility supplies eligibility for a cost claim, not the gain itself.

For intervention selection, we use a stricter workload-level objective: restore every baseline deviation without creating a new one. A selector may abstain; ordinary reference fallback remains a baseline. Its cost includes observations, probes, selection and final verification. Reproduction, restoration, causal localization and deployed performance are therefore evaluated separately.

3 Study Design and Evidence Scope

3.1 Systems, endpoints and exposure

Each study retains its own protocol, tasks and comparator. The target/draft pair is Qwen3-8B/0.6B Q8_0, except the 358H adapter's symmetric INT4 OpenVINO draft. Comparisons process one request at a time. Hardware, backend, OS and quantization jointly vary; no hardware-only causal comparison follows. Historical settings are in Appendix K and RTX V4 in Appendix Q. RTX's original direct-ID OS build is 26200; follow-ups use 26300.

Complete-output equality includes an earlier terminal EOG or the 128-ID cap. B570 untruncated native captures retain 1–2 committed post-cap IDs separately, excluded from contract equality. Stop markers are checked. Each result selects a complete attempt; interrupted attempts remain separate and are never concatenated.

B570 intervention uses 24 exposed tasks. RTX intervention uses 19 exposed HumanEval tasks and 24 MBPP tasks selected by a frozen static exposure audit before their execution. No-known-exposure concerns retained project records, not training data. The later bridge and offline analysis reuse exposed evidence. Prediction arrays precede native comparison; RTX records remote readback and B570 local seals. Neither establishes independently trusted absolute time. Task selection and exclusions remain fixed. Three repetitions assess stability; they are not independent sampling units.

Prior raw audits check source-bound arrays, traces, shams, handoffs and effective settings. This revision adds CPU checks of compact exports, not new model execution. The anonymous package exposes arithmetic; the author ledger separates provenance and access limits. Omitted weights/binaries and opaque sampler state remain outside this verification.

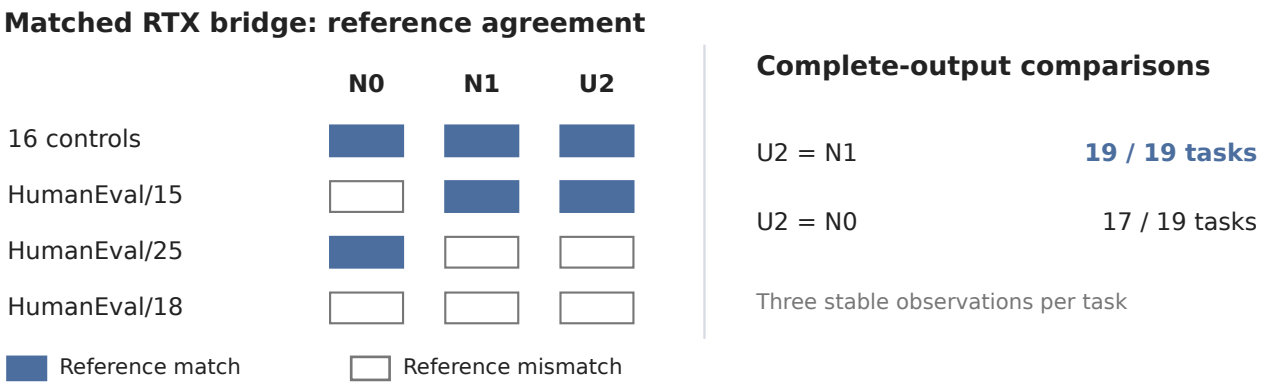


Figure 1. The fresh RTX bridge separates reproduction from reference compatibility. Ordinary target-only U_2 reproduces native intervention N_1 on every task, including /15's restoration and /25's new deviation. The other 16 controls match the reference in all conditions; /18 retains the same non-reference output. Identical match/mismatch labels alone do not imply identical output arrays; the right-hand comparisons use complete arrays and stop boundaries.

3.2 Cost boundaries and estimators

RTX V4 takes each condition's median of three times per prompt, then the prompt GM of reference/candidate ratios. Its 10,000 paired-row bootstrap draws use seed 2026100204, frozen order and linear .025/.975 percentiles. Intervals condition on selected prompt rows, observed medians and configuration; they exclude exposure-selection, population and full thermal/session uncertainty. Cost eligibility requires complete reference-ID agreement and stable reference repeats.

358H takes the median of three paired ratios before the prompt GM; total-time accounting remains separate. M4 runtime uses condition medians and its own frozen random streams. Historical parsed-text estimators, measurement deviations and energy domains remain in the supplement. Later Q gates do not certify historical H timing.

All headline latency measurements exclude loading and initial prefill. RTX V4 charges post-prefill sampler/state work through slot release, including drafting, verification, rollback, synchronization and logging. M4 and 358H retain their separately stated boundaries. No common system-energy boundary exists across platforms; no runtime intervention energy result is claimed.

Trajectory extents, score descriptors, fixed-mode opportunity and conditional bounds are post-study analyses, not new observations or preregistered primary estimates. Historical M4 Spec-Bench (Xia et al., 2024) and HumanEval (Chen et al., 2021) campaigns remain background to its separate runtime study.

4 Prediction Before and After Controlled Intervention

4.1 Distinct interventions and input contracts

The two studies test frozen predictions of complete native outputs under distinct interventions. Table 1 aligns configurations and information boundaries; the supplement maps labels and retains all paired outcomes and earlier tests.

B570 changes target prefill UB1-to-UB128 and includes the protocol's first-pending sample/state handoff; target continuation and draft remain UB1. RTX changes target post-prefill continuation UB2-to-UB1, keeping target prefill, draft UB2, $k = 2$ and caller grouping fixed. Both use same-boundary unchanged-capacity shams. Native paths receive prompt, assets and fixed parameters, use their own live draft, and receive no prediction tape, reference IDs or future native answers. The RTX description's separately orchestrated proposal/target workers share pinned numerical routines with native; this is not an independent numerical implementation.

4.2 B570: restoration on the tested exposed cohort

The 24-task intervention cohort contains four previously exposed diagnostic tasks and 20 stable controls. Baseline prediction/native equality is 72/72 and intervention equality is 72/72. Reference agreement changes from 60/72 to 72/72: HumanEval/134, /42, /145 and /138 recover in all three observations, with no new deviation among the controls. Prediction and native execution are separately checked against the complete capped arrays, not their common prefixes.

The intervention restores this exposed cohort, without isolating a pure KV factor: the first pending token is sampled

Table 1. Distinct controlled studies. UB denotes capacity, not observed graph occupancy. Counts are complete arrays/endpoints over three stable observations per task; cohorts are not pooled into a general success rate. Neither intervention has a performance or energy measurement.

Contract	B570 / Vulkan	RTX / CUDA
Cohort	24 exposed tasks: 4 diagnostic, 20 stable controls	19 exposed HumanEval; 24 exposure-audited MBPP
Baseline	Target prefill/continuation UB1, draft UB1	Target prefill/continuation UB2, draft UB2
Changed stage	Target prefill UB1→UB128 plus protocol first-pending sample/state handoff	Target continuation UB2→UB1 after UB2 prefill; first-pending sample/state handoff
Fixed path	Target continuation UB1, draft UB1, $k = 2$	Target prefill UB2, draft UB2, $k = 2$, caller grouping
Prediction inputs	Target-only prompt/assets/configuration; no draft or proposals	Own proposal worker and target worker; shared pinned numerical routines
Native inputs	Own live draft; no sealed prediction inputs	Own live draft; no H tapes/reference IDs
Sham / seal	Same-boundary UB1→UB1 / local seals	Same-boundary UB2→UB2 / remote readback before native
P_0/N_0; P_1/N_1	72/72; 72/72	57/57; 57/57 (old), 72/72; 72/72 (new)
$N_0/R \to N_1/R$	60/72→72/72	51/57→51/57; 66/72→66/72

and accepted in the source, then KV, pending logits, positions and a post-accept sampler clone are transferred. The draft-free predictor is a simpler reproducer. Earlier sampled forks and their causal limits remain in the supplement.

4.3 RTX: prediction closes while the recovery target fails

The fixed candidate is evaluated on 19 exposed HumanEval tasks and 24 statically selected MBPP tasks with no known prior execution exposure within the audited project records. For each cohort, both predictions match every complete native array and endpoint: 57/57 per condition on HumanEval and 72/72 per condition on MBPP. All 43 tasks are repeat-stable. The two prediction comparisons are not 258 independent tasks.

Figure 2 shows the paired reference effects. HumanEval/15 changes from mismatch to match; /18 remains divergent; /25 changes from match to mismatch. Thus HumanEval reference agreement remains 51/57 arrays, or 17/19 tasks, before and after. On MBPP, 22 tasks remain matching and /463 and /304 remain divergent, giving 66/72 arrays in each condition. Prediction closure therefore coexists with failure of the predefined reference-recovery plus no-new-regression goal.

Persistent reference deviation alone says nothing about output no-op. HumanEval/18 and MBPP/463 retain identical complete native outputs and EOS endpoints. MBPP/304 changes its 47-ID output at zero-based index 33 (488 to 481), yet remains divergent. The appendix provides full endpoint and first-difference witnesses. Across RTX the native/reference total is 117/129 in each condition: 43 tasks with three stable observations, not a population success rate.

Target-continuation UB2-to-UB1 alone did not reproduce historical global-UB1 reference agreement under this controlled configuration. The historical global option also changed prefill and draft, and historical native defaults were effectively 12 threads despite a 24-thread description; this study explicitly uses 24. Its effective baseline/sham qualification (G0) contains 75 newly checked native arrays at 24 threads and 45 unchanged prediction-worker (H) arrays reused, not 120 new 24-thread outputs. These differences preclude assigning necessity to prefill, drafting or their interaction.

5 A Matched Simple-Baseline Test

5.1 Retrospective lead and fresh comparison

A post-study comparison linked historical target-only UB2 outputs to the RTX intervention: $U_2 = N_1$ in 57/57 arrays, while $U_2 = N_0$ in 51/57. Prompt hashes matched, but historical native commands selected 12 threads and the intervention used 24. We therefore froze a new matched study on the same 19 exposed HumanEval tasks. It is a fresh confirmation of an exposed-cohort hypothesis, not a new discovery set.

The study executes three fresh-process repetitions of N_0, N_1, U_2: 171 runs. It fixes target/draft assets, b10839 runtime, the qualified native frontend and relevant numerical threads at 24, rotates condition order, and uses one GPU queue without process/KV reuse. Twelve engineering checks precede formal runs; there are no failed attempts or retries. Prior instrumentation qualification is reused.

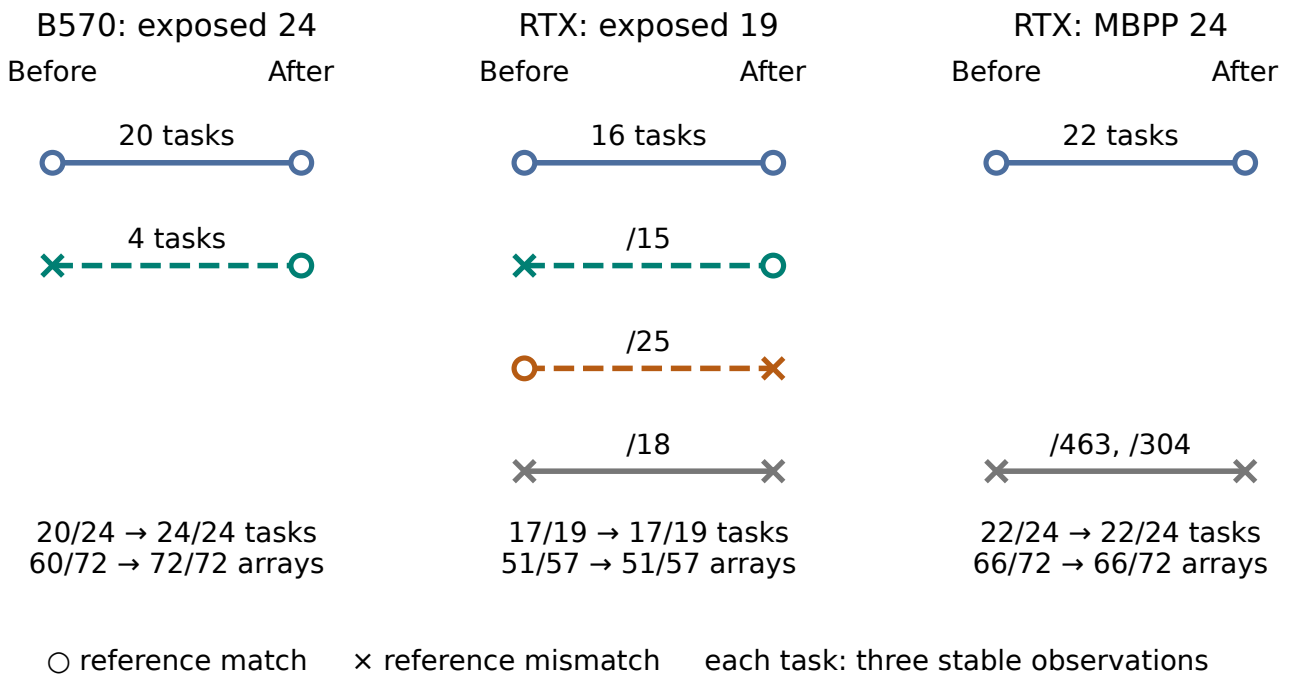


Figure 2. Task-level paired reference effects. B570 restores four diagnostic tasks with no new deviation among 20 controls. RTX HumanEval restores /15 and newly deviates on /25, leaving 17/19 tasks (51/57 repeated arrays) matching in both conditions; MBPP is shown separately. Each task has three stable observations. Equality compares the full protocol-selected generated-ID array and endpoint (inclusive earlier EOG or cap, with raw overshoot retained); mismatch is not functional failure. Equal aggregate agreement can conceal paired restoration and new deviation.

Table 2. Fresh matched RTX bridge. Full stop boundary is (length, EOS/limit reason, terminal ID). Task equality requires all three observations to agree.

Pair	Arrays	Stop boundary	Tasks
U_2/N_1	57/57	57/57	19/19
U_2/N_0	51/57	54/57	17/19
N_0/N_1	51/57	54/57	17/19

U_2 uses the released CLI with its own prefill and single-token continuation, without draft, state transplant or supplied output. Caller grouping is observed, not modified. Formal outputs are newly generated; historical arrays are comparison evidence.

5.2 Simple reproduction and mixed restoration

$U_2 = N_1$ on all 19 tasks and all 57 complete arrays and stop boundaries (Table 2). Each condition is repeat-stable on every task. Both changed tasks agree with U_2: /15 changes from 25 to 28 IDs and restores reference; /25 retains 75 IDs but changes index 4 and becomes divergent. /18 retains the same 69-ID divergent output. Thus ordinary target-only execution captures the nontrivial changes as well as the 17 no-ops. Predicting $N_1 = N_0$ matches 17/19 tasks; U_2 matches 19/19.

The frozen endpoint field stores EOS/limit reason. A supplementary stop boundary adds committed length and terminal ID. Reason agreement is 57/57 for every pair, but full-boundary agreement for N_0/N_1 and U_2/N_0 is 54/57 because /15's lengths differ. Array-plus-boundary agreement is unchanged. All 60 recorded uncommitted target samples remain outside committed-output equality and inspectable separately.

Fresh N_0, N_1 each match their historical counterpart 57/57; fresh 24-thread U_2 matches historical 12-thread U_2 57/57. This removes unmatched thread settings from the fresh comparison but is not a thread-causality experiment. The simpler path suffices for N_1 on this cohort; it does not establish general equivalence or an advantage for the complex predictor. Reference agreement remains 17/19 in every condition, with different failing task identities.

Table 3. Offline selection replay after common reference/baseline acquisition. Counts are available historical record equivalents, not measured time or new inference.

Workload	Policy	Records	Decision
RTX19	Enumerate	114	Abstain
RTX19	Proxy + verify	114	Abstain
B57024	Verify native	72	Select N_1
B57024	Proxy + verify	144	Select N_1

5.3 What the traces explain

On /15, /25, /18 and /31, native initial source KV and pending logits match between N_0, N_1. At the two first divergences, target candidate ranks flip. Native continuation observes occupied-token counts 1/2 for N_0 and 1 for N_1; ordinary U_2 continuation batch dumps contain one token despite UB2 capacity. Its graph tensors and live KV/logits are unobserved. These witnesses are consistent with shape-sensitive numerical execution, without proving identical later state or a unique operation. The separate supplement retains the scores, boundaries and capture limits.

6 Does Prediction Improve Intervention Selection?

The bridge motivates a CPU-only feasibility test: can a selector choose one configuration-level action that restores the fixed reference across a workload without new deviations more efficiently than direct alternatives? Reference fallback is a baseline; retaining speculation needs an operational benefit, unmeasured here.

We inspect two candidates on existing exposed evidence. A stage/state-fingerprint rule would compare aligned live state and choose an intervention at the earliest differing stage. It is not implemented: required reference/U_2/later native state is missing, and a stage difference alone supplies no repair direction. The same prescribed continuation change accompanies a restoration, a regression and no-ops. Post-intervention scores cannot serve as free pre-decision features.

The second candidate is generic paid proxy screening followed by actual-action verification. An offline query interface reveals retained outcomes only after charging their acquisition. The policy sees handles and paid replies without task/platform names, reference tokens or unqueried labels. Accepted actions require verification; unknown counterfactuals remain unknown. Both orders of the two RTX actions and the sole B570 action are compared with direct enumeration; no per-task mixture is selected after seeing outcomes.

On RTX, screening N_1 through its proposal-conditioned predictor and then testing U_2 leads to abstention, as do both enumeration orders. On B570, screening through T_1 and then verifying N_1 selects the same sole candidate as direct verification but adds 72 proxy records. These are retrospective acquisition equivalents, not new model executions, independent cases or latency measurements. The already-exposed MBPP supplement has two persistent failures and no observed repairs; its entire U_2 counterfactual remains unknown.

For a sole accepted action, enumeration costs $O + N + V$ and screening costs $O + P + A + N + V$, where O is common observation, P proxy work, A extra selection/setup and V verification outside the candidate execution. Screening adds $P + A$. For a rejected action, savings require $P + A + (V_P - V_E) < N$, at a comparable success and false-rejection contract. RTX prediction retains substantial numerical work; no matched timing establishes that inequality. Cached decisions can also benefit enumeration. No debugging-time or deployed-cost saving follows.

The result is a scoped negative feasibility finding: neither candidate supplies a supported distinct selection capability here. Ten integrity/charging tests pass, without held-out validation. The supplement retains ledgers, missingness and cost cases.

7 Qualification and Optimization Limits

7.1 RTX: matched qualification and complete cost

The historical V4 cost arm uses the same 19 tasks as its prediction arm but 171 fresh observations: target-only UB1, batched $k = 2$ UB2 and serialized $k = 2$ UB1. One frozen timer executable, model/runtime, input, endpoint and balanced order schedule cover all three paths; each condition occupies each order position 19 times. The global native micro-batch option also propagates to draft, making this a whole-path configuration comparison, not an isolated target intervention.

The fresh ID gate blocks batching at 51/57. Serialization qualifies at 57/57, but its prompt-median target/candidate GM is 0.677× [0.662, 0.692], with 0/19 prompts faster (Table 4). Reciprocal relative GM latency is 1.476×. The experiment excludes the measured serialized configuration as a latency optimization and establishes no fast qualified alternative. Blocked batching supplies no admissible speedup or CI.

Appendix Q retains every prompt-condition median and absolute latency descriptors. Median prompt times are 1.607s for target and 2.332s for serialization; their ratio is not the paired GM. The earlier 30-prompt result, 0.646× [0.630, 0.660], uses a different cohort and no

Table 4. RTX V4 matched qualification and cost: 171 fresh runs on 19 prompts × three × three paths. Target/serialized use UB1, batching UB2; speculation uses $k = 2$. Speedup = target/candidate latency. Only the blocked batched row has null speedup/CI. Serialization is faster on 0/19 prompts.

Path	ID gate	Speedup [95% CI]
Target	Reference	1.000
Batched	51/57 blocked	—
Serialized	57/57 qualified	0.677 [0.662, 0.692]

batched condition; the two estimates are not an optimization trend.

The native timer charges per-request sampler/state work after synchronized prefill, drafting, target validation, acceptance/correction/bonus, KV rollback, synchronization and related logging/response through slot release. Loading, prefill and teardown are excluded. Observer controls check output stability, while clock probes bound clock-call overhead only; condition-dependent debug logging need not cancel. Thus the conclusion concerns instrumented post-prefill request latency, not logging-free deployment, throughput or energy. The bootstrap conditions on the selected rows and observed medians as stated in Section 3.

7.2 358H: a bound on draft-only optimization

The closed 358H Phase-3 runner directly qualifies CPU and NPU drafting at 90/90 pairs per path, yet both are slower than target-only: CPU 0.734× [0.693, 0.771], NPU 0.352× [0.333, 0.369], with zero of 30 prompts faster. Isolated NPU drafting is faster than CPU at tested contexts, but the integrated synchronous OpenVINO region combines full-prefix replay/reconstruction and proposal. Its 58.35% share cannot be separated into the two costs (Figure 3). This is an interface result, not a general NPU limitation.

Measured committed progress is $C = 2.4631$ CPU and 2.4584 NPU, including correction/bonus. Complete target work consumes 87.34% of CPU-path time; residual orchestration is 0.04% CPU and 0.02% NPU. Processed-total accounting gives $(D_{\mathrm{real}}, R_{\mathrm{target}}, X) = (0.4304, 2.9767, 0.00124)$ CPU and $(4.1251, 2.9435, 0.00130)$ NPU. Their 0.7227× and 0.3477× total-time ratios differ from prompt GMs.

With baseline, target work and progress fixed, eliminating draft/residual gives pooled-total ceilings $C/R_{\mathrm{target}} =$ 0.8275× CPU and 0.8352× NPU. The original prompt estimator has its own bound. Let B_{ij} and T_{ij} be matched baseline and complete target time for prompt i, repetition j. Nonnegative remaining cost and monotonicity of the median and GM give

$$\mathrm{GM}_i \,\mathrm{median}_j\, S'_{ij} \le \mathrm{GM}_i \,\mathrm{median}_j (B_{ij}/T_{ij}). \quad (3)$$

All 90 rows per path yield ceilings 0.8372× CPU and 0.8448× NPU. Thus overhead-only savings cannot reach parity under either estimator. Even with other cost zero, a uniform target-work reduction must be at least 16.28%/15.52% to make the prompt-GM ceiling reach one. These are post-study necessary budgets, not achieved savings; changed progress or target execution requires new qualification and measurement. Appendix P gives full precision and per-prompt limits. The persistent-state/wider-target candidate stops at qualification, with no performance or energy result.

CPU restore/reconcile correctness could not be established under the tested state representation; the NPU state-capable path was unavailable with the frozen dynamic-shape IR. Separate correctness-only controls reproduce UB1/UB2 greedy divergence on the target even with exact matching prefixes, and proposal replay reproduces speculative divergence without live NPU draft inference. These controls identify no unique floating-point operation. The implementation motivated by cost remains blocked by output compatibility; neither device is proved incapable of retaining state.

8 Runtime Control under Qualified Fixed Modes

The separate M4 runtime study tests whether continuation control improves a qualified speculative path. It uses its own frozen policy, direct-ID gate and cost boundary; historical M4 timing is background rather than its qualification evidence.

8.1 A prospective cost guard on M4

Guard tests a frozen rule against target-only and fixed speculation, using Qwen3-8B/0.6B Q8_0 on llama.cpp Metal, fixed $k = 2$, greedy generation and a 128-token cap. It starts on Always-Spec's path and may fall back permanently to target-only. At eligible completed rounds it divides all charged generation cost by committed progress, including the initial seed and correction/bonus; rejected proposals do not count. It continues when cost per token is at most $(1-m)R_{\mathrm{ref}}$, otherwise falling back after reconciling invalid suffix state. Zero progress also triggers fallback.

Calibration freezes the target reference and selects margin 0 with decisions starting after 8 rounds. The controller receives no task labels, future length or request-specific oracle. The post-prefill timer charges control and transition costs; Appendix T retains the full rule, grid, tie-breaks and timing boundary.

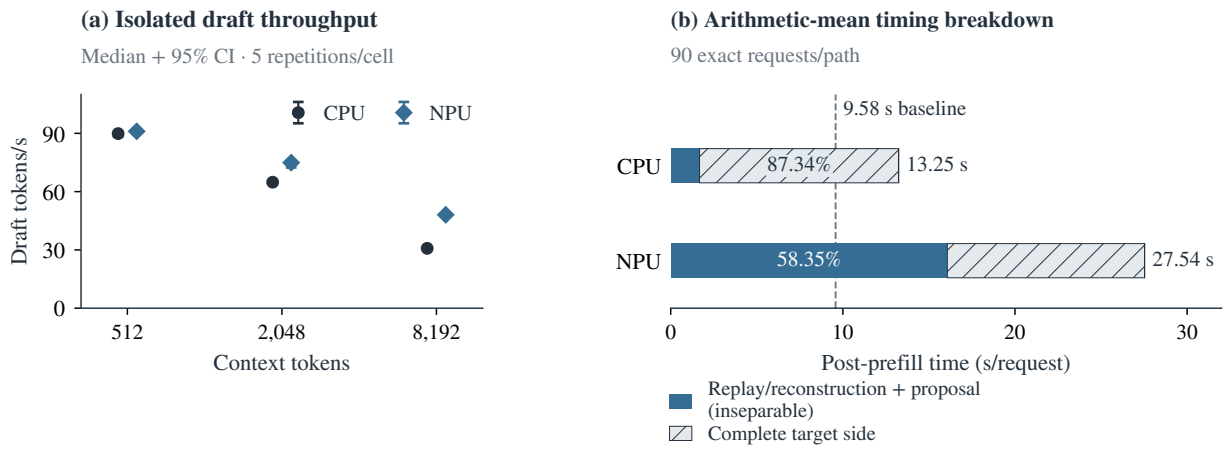


Figure 3. **Isolated draft advantage and complete 358H cost.** A: five-repeat steady-state draft throughput with repetition-bootstrap intervals. B: arithmetic mean post-prefill seconds over 90 exact observations per path, against the shared baseline. NPU's 58.35% region combines full-prefix replay/reconstruction and proposal; it is inseparable. CPU complete target work is 87.34%. These bars differ from the prompt-GM speedups in Section 7.2; persistence savings are unmeasured.

8.2 Held-out evaluation

Historical outcomes informed the design; prospective means policy freeze before reserved outcomes. Exposure exclusion and deterministic selection give disjoint calibration, qualification and held-out splits of 40, 16 and 60 prompts, balanced across HumanEval and Spec-Bench. Policy and reference precede qualification/formal measurement; the appendix retains the audited freeze-hash metadata defect. Always-Spec and Guard each qualify 48/48 direct-ID pairs.

The held-out study runs 3 repetitions of three conditions on 60 prompts (540 observations), randomizing condition order within prompt/repetition. Both speculative paths preserve 180/180 complete direct-ID sequences. Prompt condition medians precede latency ratios and the GM; 10,000 prompt-bootstrap draws use seed 20260924. The supplement retains exact estimator, exposure and metadata-recovery details.

Guard is faster than target-only, but fixed speculation has the higher speedup (Table 5). A post-study paired comparison gives Guard/Always-Spec latency 1.0271 [1.0130, 1.0436], entirely above parity. Guard is faster on 24 prompt medians, Always-Spec on 36, with 0 ties. Fallback occurs in 30.6% of 180 executions, measuring activity rather than beneficial decisions. Guard/oracle is 1.0278×; this hindsight oracle chooses the lower target-only or Always-Spec median, not an achievable runtime policy.

Reconstructing all original medians finds target-only faster than Always on only 2/60 prompts, both Spec-Bench. Always/oracle is 1.0007×. Choosing the better of these two measured fixed modes after seeing outcomes therefore offers about 0.069% GM latency reduction from Always, computed from the full-precision ratio. Guard falls back in all six executions of those two prompts but still loses to target-only on their medians. The family-stratified sensitivity analysis also leaves Guard slower (Appendix P). These post-study comparisons estimate neither transition cost nor opportunity for arbitrary within-request switching.

Table 5. M4 ratios of paired prompt medians. Both speculative policies preserve 180/180 direct-ID sequences. Speedup and relative latency have opposite favorable directions. The oracle selects the lower of two measured fixed-mode medians with hindsight. Guard/Always is post-study secondary; values above one mean Guard is slower.

Ratio (×)	GM	95% prompt CI
Speedup: above one favors speculation		
Target / Guard	1.3114	[1.2487, 1.3787]
Target / Always-Spec	1.3469	[1.2867, 1.4088]
Relative latency: lower is better		
Guard / offline oracle	1.0278	[1.0137, 1.0446]
Post-study paired secondary latency ratio		
Guard / Always-Spec	1.0271	[1.0130, 1.0436]

Guard preserves IDs but supplies no incremental benefit over Always-Spec. Fallback activity does not establish beneficial decisions or risk reduction. No runtime energy claim is made. Policy transfer and stable small-stratum effects remain unestablished; the archived policy-freeze defect and original stopped pipeline markers are documented

separately.

9 Implications for System Evaluation

The evidence supports three bounded engineering lessons.

Successful reproduction does not establish predictor utility. B570's target-only program provides a simpler reproducer on its tested paths. RTX's original proposal-conditioned program matches native outputs, but ordinary U_2 also reproduces the intervention on all 19 bridge tasks. A baseline that is sufficient for N_1 is not sufficient for every N_0 output. Claims of necessary execution history or reduced diagnosis cost require additional evidence beyond either finite equality result.

Paired outcomes determine whether a repair is useful. RTX's equal before/after agreement hides a restoration and a regression. Both matter to a workload-global reference contract. The plain reference gate already rejects that intervention; a complete output forecast adds descriptive information without a demonstrated selection benefit. B570's restoration is useful finite evidence, but screening its sole qualified action before verifying it adds work. A state difference may motivate a hypothesis without identifying an effective repair.

Optimize within a measured opportunity. RTX serialization loses despite qualification. On 358H, overhead-only changes cannot reach parity under the fixed target-work/progress premise. On M4, the hindsight opportunity between two measured fixed modes is small and the tested guard is slower than Always-Spec. These findings favor inspecting complete work and strong fixed baselines before adding control. They do not rule out a different backend, target execution, progress rate or within-request policy.

The current evidence supports bounded characterization rather than a general construction rule. A future method would need a concrete operational benefit, decision inputs available before observing the intervention outcome, and a cost comparison against direct execution or enumeration.

10 Related Work

Performance and heterogeneous execution. Speculative decoding amortizes target computation under its algorithmic assumptions (Leviathan et al., 2023). Prior work already documents verification and workload limits: Liu et al. (2026b) study verification dominance and batch effects in vLLM; Kim et al. (2026) account for conditional execution cost and runtime control; Lyu et al. (2026) derive edge–cloud profitability conditions. On consumer Apple silicon, Chordiya (2026) reports gains and slowdowns across five backend configurations, diagnoses ineffective batch-parallel verification in one quantized Metal path, tests output agreement, and observes target-only batch-shape argmax changes. Slowdown and correctness gating have already been studied, as has heterogeneous placement: Dovetail, DuoDecoding, and Mirror-SD study GPU/CPU or GPU/NPU paths (Zhang et al., 2025; Lv et al., 2025; Bhendawade et al., 2025); Ruiz y Mesa et al. (2026) combine an analytical cost model with compiler-oriented CPU/GPU partitioning, and Lever jointly optimizes draft, verification, and CPU/NPU execution for a smartphone target (Wang et al., 2026). State reuse has prior art in compressed draft KV memory (Yuan et al., 2026). Our 358H study measures one interface that combines replay and proposal; it establishes no general placement or persistence result.

Numerical paths and output contracts. The separate appendix compares the closest prior questions. Numerical sensitivity, deterministic inference construction, correctness forensics and cost gating are established prior work. LLM-42 uses fixed-shape verification and KV repair to construct deterministic inference (Gond et al., 2026); our tests forecast finite native paths and do not establish that guarantee. Our controlled studies retain complete output forecasts and paired reference effects, but the new RTX bridge also establishes a strong ordinary target-only alternative. We therefore make no general prediction-method or efficiency claim. MarginGate gates deterministic verification by a calibrated logit margin and repairs the current KV column (Chu et al., 2026); our post-outcome score observations supply neither that policy nor a competing repair guarantee. Fixed-prefix precision controls and bidirectional cache transplantation show how replay divergence can follow live KV state (Lorup, 2026). Our shared prefixes do not supply such a causal transplantation test. Integer Alibi localizes cross-kernel differences under verified shared-operand, nonoverflowing INT8 accumulation assumptions (Chen, 2026); those assumptions are not established for our Q8_0 runtime dispatch. We identify no unique arithmetic operation. Qin et al. (2026) formalize and verify fixed-deployment determinism through engine invariants and relational kernel contracts, including prefill/decode and KV-cache changes. Their bitwise-logit specification is stronger than our finite generated-ID comparisons. We forecast tested paths that can remain reference-incompatible; we supply neither an engine proof nor a deterministic implementation guarantee.

Adaptive speculation and the study boundary. TurboSpec adjusts speculation for serving goodput (Liu et al., 2025); BanditSpec selects configurations online (Hou et al., 2025); ACL HeteroSpec adapts verification from contextual entropy (Liu et al., 2026a). Adaptive control is established prior work. Our M4 Guard is a qualified runtime control

test, not a new policy advantage: it beats target-only, but fixed speculation is faster in the post-study paired secondary comparison. Our measurements test prospective direct-ID qualification, frozen trajectory prediction, sampled intervention effects and realized cost on qualified paths.

11 LIMITATIONS

The studies use a narrow Qwen3 model pair, greedy generation and finite endpoints. Hardware, backend, OS and quantization jointly vary. The results are not a hardware ranking, a population failure estimate, functional pass@k evaluation, or a guarantee for production serving. Single-request execution does not characterize dynamic multi-request batching.

Exposure differs by study. B570 intervention and the RTX bridge reuse known tasks. RTX MBPP was statically exposure-audited before its original intervention study, but is exposed for the later offline analysis. Fresh confirmation is not unseen-task validation; repeats and token positions are correlated. Unknown U_2 outcomes on MBPP and B570 remain unknown; separate interventions cannot be combined into an executed factorial design.

Prediction compares generated IDs and endpoints, not complete numerical state. B570's prefill change includes first-pending sampling and state handoff. RTX shares numerical libraries across paths; live U_2 internals and complete later divergence-point KV are unavailable. Neither observed shape/score differences nor equal output prefixes establish a unique causal stage, kernel, minimal state or repair direction. The tested RTX recovery/no-regression target fails despite reproduction closure.

Hashes bind supplied bytes, not independently trusted execution or time. Offline review does not execute omitted weights/binaries or prove opaque sampler-state identity; its scopes are documented separately.

Intervention and bridge studies have no qualified latency, energy or VRAM benefit. Historical RTX timing includes logging and changes multiple stages; it cannot price the new intervention or proxy. The 358H region combines replay/reconstruction and proposal, and some runtime raw traces are unavailable. Its ceilings fix baseline, target work and progress; necessary reduction budgets are not achieved savings. Blocked paths establish no architectural impossibility.

M4's paired secondary contrast and two-fixed-mode oracle are post-study analyses. The oracle is not a bound on arbitrary within-request policies. Intervals condition on selected prompts, observed medians and configurations, omitting full session/recalibration uncertainty. Energy domains differ and runtime energy is unmeasured. The documented metadata defects and stopped historical attempts remain part of the evidence record.

The offline selector study examines two bounded candidates on exposed evidence. Its negative decision is not an impossibility theorem. Missing timing prevents a quantitative rejection of every cheap-proxy regime, while missing aligned state prevents testing a general stage-based selector. No automatic causal diagnosis or prospective selector validation is claimed.

12 CONCLUSION

Controlled output prediction, reference restoration and optimization produce different verdicts in the tested speculative-decoding stacks. B570's intervention restores its exposed cohort; RTX's predicts correctly while repairing one task and introducing another deviation. A fresh matched study shows that ordinary target-only execution already reproduces the RTX intervention on all 19 tested tasks. Offline selection adds no demonstrated advantage over simple baselines. Separate measurements show qualified RTX slowdown, conditional 358H limits and M4 control losing to fixed speculation. Together these results support explicit output contracts, paired effects and complete cost accounting, while narrowing claims of general prediction, repair and control capability.

REFERENCES

Bhendawade, N., Nishu, K., Kundu, A., Bartels, C., Cho, M., and Belousova, I. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161v2*, 2025. URL https://arxiv.org/abs/2510.13161v2.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021. URL https://arxiv.org/abs/2107.03374v2.

Chen, T.-R. The integer alibi: Localizing cross-kernel divergence in INT8-quantized LLM inference. *arXiv preprint arXiv:2608.13756v1*, 2026. URL https://arxiv.org/abs/2608.13756v1.

Chordiya, P. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026. doi: 10.48550/arXiv.2607.17283. URL https://arxiv.org/abs/2607.17283.

Chu, K., Zhou, Y., and Zhang, W. MarginGate: Sparse margin-triggered verification for batch-invariant LLM inference. *arXiv preprint arXiv:2605.30218v1*, 2026. URL https://arxiv.org/abs/2605.30218v1.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026. URL https://arxiv.org/abs/2601.17768v2.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 24045–24079. PMLR, 2025. URL https://proceedings.mlr.press/v267/hou25h.html.

Kim, S., Kim, J., Cho, Y., and Seo, J. Runtime-controlled evaluation of speculative decoding methods. *Electronics*, 15(14):3179, 2026. doi: 10.3390/electronics15143179. URL https://www.mdpi.com/2079-9292/15/14/3179.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, S., Ye, Y., Zhu, Q., Cao, Z., and He, Y. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12930–12947. Association for Computational Linguistics, July 2026a. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066v3*, 2025. URL https://arxiv.org/abs/2406.14066v3.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026b. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lorup, A. B. Stage-replay divergence follows the KV cache: Fixed-prefix precision controls and bidirectional cache transplantation. *arXiv preprint arXiv:2607.28495v1*, 2026. URL https://arxiv.org/abs/2607.28495v1.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025. URL https://arxiv.org/abs/2503.00784.

Lyu, Y., Irukulapati, B., and Champati, J. P. Speculation at a distance: Where edge-cloud speculative decoding actually pays off. *arXiv preprint arXiv:2606.25091v2*, 2026. URL https://arxiv.org/abs/2606.25091v2.

Qin, J., Du, A., Zhang, D., Lentz, M., and Zhuo, D. Vosti: Specifying, implementing, and verifying deterministic

llm inference. *arXiv preprint arXiv:2609.38981v1*, 2026. URL https://arxiv.org/abs/2609.38981v1.

Ruiz y Mesa, A., Korol, G., Riesterer, M., Cardoso de Lima, J. P., and Castrillon, J. Compiler-assisted speculative sampling for accelerated LLM inference on heterogeneous edge devices. In *8th Workshop on Accelerated Machine Learning (AccML), co-located with HiPEAC*, January 2026. URL https://accml.dcs.gla.ac.uk/papers/2026/8th_AccML_paper_8.pdf.

Wang, T., Li, F., Sun, Y., Gao, W., and Ren, J. Lever: Speculative LLM inference on smartphones. *arXiv preprint arXiv:2605.16786*, 2026. doi: 10.48550/arXiv.2605.16786. URL https://arxiv.org/abs/2605.16786.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https://aclanthology.org/2024.findings-acl.456/.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501v2*, 2025. URL https://arxiv.org/abs/2506.09501v2.

Yuan, T., Liao, C., and Wen, Z. Strong drafts need compact memories: Long-context speculative decoding with compressed KV cache. *arXiv preprint arXiv:2608.30252*, 2026. URL https://arxiv.org/abs/2608.30252. EMNLP 2026 Findings.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025. URL https://arxiv.org/abs/2412.18934v2.